

Informe Laboratorio 5

Sección 3

Alumno Felipe Ormazábal
e-mail: felipe.ormazabal1@mail.udp.cl

Noviembre de 2025

Índice

Descripción de actividades	3
1. Desarrollo (Parte 1)	6
1.1. Códigos de cada Dockerfile	6
1.1.1. C1	6
1.1.2. C2	6
1.1.3. C3	6
1.1.4. C4/S1	7
1.2. Creación de las credenciales para S1	7
1.3. Tráfico generado por C1, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)	7
1.4. Tráfico generado por C2, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)	8
1.5. Tráfico generado por C3, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)	8
1.6. Tráfico generado por C4 (iface lo), detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)	9
1.7. Tipo de información contenida en cada uno de los paquetes generados en texto plano	9
1.7.1. C1	9
1.7.2. C2	9
1.7.3. C3	10
1.7.4. C4/S1	10
1.8. Diferencia entre C1 y C2	10
1.9. Diferencia entre C2 y C3	10
1.10. Diferencia entre C3 y C4	10
2. Desarrollo (Parte 2)	11
2.1. Identificación del cliente SSH con versión “?”	11
2.2. Replicación de tráfico al servidor (paso por paso)	11
3. Desarrollo (Parte 3)	11
3.1. Replicación del KEI con tamaño menor a 300 bytes (paso por paso)	11
4. Desarrollo (Parte 4)	12
4.1. Explicación OpenSSH en general	12
4.2. Capas de Seguridad en OpenSSH	12
4.3. Identificación de que protocolos no se cumplen	13

Descripción de actividades

Para este último laboratorio, se solicita trabajar con Docker y el protocolo SSH, a fin de poder entender el concepto de criptografía asimétrica y firmas digitales.

Para lo anterior deberá:

- Crear 4 contenedores en Docker por medio de un DockerFile, donde cada uno tendrá el siguiente SO: Ubuntu 16.10, Ubuntu 18.10, Ubuntu 20.10 y Ubuntu 22.10 a los cuales se llamarán C1, C2, C3 y C4 respectivamente.
El equipo con Ubuntu 22.10 también será utilizado como S1.
- Para cada uno de ellos, deberá instalar el cliente openSSH disponible en los repositorios de apt, y para el equipo S1 deberá también instalar el servidor openSSH.
- En S1 deberá crear el usuario “**prueba**” con contraseña “**prueba**”, para acceder a él desde los clientes por el protocolo SSH.
- En total serán 4 escenarios, donde cada uno corresponderá a los siguientes equipos:
 - C1 → S1
 - C2 → S1
 - C3 → S1
 - C4 → S1

Pasos:

1. Para cada uno de los 4 escenarios, solo deberá establecer la conexión y no realizar ningún otro comando que pueda generar tráfico (como muestra la Figura). Deberá capturar el tráfico de red generado y analizar el patrón de tráfico generado por cada cliente. De esta forma podrá obtener una huella digital para cada cliente a partir de su tráfico.

Indique el tamaño de los paquetes del flujo generados por el cliente y el contenido asociado a cada uno de ellos. Indique qué información distinta contiene el escenario siguiente (diff incremental). El objetivo de este paso es identificar claramente los cambios entre las distintas versiones de ssh.

2. Para poder identificar que el usuario efectivamente es el informante, éste utilizará una versión única de cliente. ¿Con qué cliente SSH se habrá generado el siguiente tráfico?

Protocol	Length	Info
TCP	74	34328 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=14
TCP	66	34328 → 22 [ACK] Seq=1 Ack=1 Win=64256 Len=0
SSHv2	85	Client: Protocol (SSH-2.0-OpenSSH_?)
TCP	66	34328 → 22 [ACK] Seq=20 Ack=42 Win=64256 Len=
SSHv2	1578	Client: Key Exchange Init
TCP	66	34328 → 22 [ACK] Seq=1532 Ack=1122 Win=64128
SSHv2	114	Client: Elliptic Curve Diffie-Hellman Key Exc
TCP	66	34328 → 22 [ACK] Seq=1580 Ack=1574 Win=64128
SSHv2	82	Client: New Keys
SSHv2	110	Client: Encrypted packet (len=44)
TCP	66	34328 → 22 [ACK] Seq=1640 Ack=1618 Win=64128
SSHv2	126	Client: Encrypted packet (len=60)
TCP	66	34328 → 22 [ACK] Seq=1700 Ack=1670 Win=64128
SSHv2	150	Client: Encrypted packet (len=84)
TCP	66	34328 → 22 [ACK] Seq=1784 Ack=1698 Win=64128
SSHv2	178	Client: Encrypted packet (len=112)
TCP	66	34328 → 22 [ACK] Seq=1896 Ack=2198 Win=64128

Figura 1: Tráfico generado del informante

Replique este tráfico generado en la imagen. Debe generar el tráfico con la misma versión resaltada en azul. Recuerde que toda la información generada es parte del sw, por lo tanto usted puede modificar toda la información.

3. Para que el informante esté seguro de nuestra identidad, nos pide que el patrón del tráfico de nuestro server también sea modificado, hasta que el Key Exchange Init del server sea menor a 300 bytes. Indique qué pasos realizó para lograr esto.

TCP	66	42350 → 22	[ACK]	Seq=2	Ack=
TCP	74	42398 → 22	[SYN]	Seq=0	Win=
TCP	74	22 → 42398	[SYN, ACK]	Seq=0	
TCP	66	42398 → 22	[ACK]	Seq=1	Ack=
SSHv2	87	Client: Protocol (SSH-2.0-C			
TCP	66	22 → 42398	[ACK]	Seq=1	Ack=
SSHv2	107	Server: Protocol (SSH-2.0-C			
TCP	66	42398 → 22	[ACK]	Seq=22	Ack=
SSHv2	1570	Client: Key Exchange Init			
TCP	66	22 → 42398	[ACK]	Seq=42	Ack=
SSHv2	298	Server: Key Exchange Init			
TCP	66	42398 → 22	[ACK]	Seq=1526	A

Figura 2: Captura del Key Exchange

- Tomando en cuenta lo aprendido en este laboratorio, así como en los anteriores, explique el protocolo OpenSSH y las diferentes capas de seguridad que son parte del protocolo para garantizar los principios de seguridad de la información, integridad, confidencialidad, disponibilidad, autenticidad y no repudio. Es importante que sea muy específico en el objetivo del principio en el protocolo. En caso de considerar que alguno de los principios no se cumple, justifique su razonamiento. Es fundamental que su análisis se base en el tráfico SSH interceptado.

1. Desarrollo (Parte 1)

1.1. Códigos de cada Dockerfile

1.1.1. C1

```
archivosvs > Cripto > lab5 > c1 > Dockerfile > ...
1 FROM ubuntu:16.10 (last pushed 8 years ago)
2
3 RUN sed -i 's/archive.ubuntu.com/old-releases.ubuntu.com/g' /etc/apt/sources.list && \
4     sed -i 's/security.ubuntu.com/old-releases.ubuntu.com/g' /etc/apt/sources.list && \
5     apt-get update && \
6     apt-get install -y openssh-client && \
7     rm -rf /var/lib/apt/lists/*
8
```

Figura 3: Dockerfile para C1 (Ubuntu 16.10)

1.1.2. C2

```
archivosvs > Cripto > lab5 > c2 > Dockerfile > ...
1 FROM ubuntu:18.10 (last pushed 6 years ago)
2
3 RUN sed -i 's/archive.ubuntu.com/old-releases.ubuntu.com/g' /etc/apt/sources.list && \
4     sed -i 's/security.ubuntu.com/old-releases.ubuntu.com/g' /etc/apt/sources.list && \
5     apt-get update && \
6     apt-get install -y openssh-client && \
7     rm -rf /var/lib/apt/lists/*
8
```

Figura 4: Dockerfile para C2 (Ubuntu 18.10)

1.1.3. C3

```
archivosvs > Cripto > lab5 > c3 > Dockerfile > ...
1 FROM ubuntu:20.10 (last pushed 4 years ago)
2
3 RUN sed -i 's/archive.ubuntu.com/old-releases.ubuntu.com/g' /etc/apt/sources.list && \
4     sed -i 's/security.ubuntu.com/old-releases.ubuntu.com/g' /etc/apt/sources.list && \
5     apt-get update && \
6     apt-get install -y openssh-client && \
7     rm -rf /var/lib/apt/lists/*
```

Figura 5: Dockerfile para C3 (Ubuntu 20.10)

1.1.4. C4/S1

```

archivosvs > Cripto > lab5 > c4 > Dockerfile > ...
1 FROM ubuntu:22.10 (last pushed 2 years ago)
2
3 RUN sed -i 's/archive.ubuntu.com/old-releases.ubuntu.com/g' /etc/apt/sources.list && \
4     sed -i 's/security.ubuntu.com/old-releases.ubuntu.com/g' /etc/apt/sources.list && \
5     apt-get update && \
6     apt-get install -y openssh-client openssh-server && \
7     rm -rf /var/lib/apt/lists/*
8
9 RUN useradd -m prueba && echo 'prueba:prueba' | chpasswd
10 WORKDIR /home/prueba
11 EXPOSE 22
12
13 CMD ["/usr/sbin/sshd", "-D"]

```

Figura 6: Dockerfile para C4/S1 (Ubuntu 22.10)

1.2. Creación de las credenciales para S1

El usuario prueba con contraseña prueba se crea mediante el siguiente comando presente en el Dockerfile del servidor:

```
RUN useradd -m prueba && echo 'prueba:prueba' | chpasswd
```

1.3. Tráfico generado por C1, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

Los paquetes clave, como Key Exchange Init y Diffie-Hellman, tienen tamaños de 1580, 1148 y 116 bytes, lo que refleja una negociación basada en algoritmos antiguos y menos opciones de cifrado. El HASSH captura los algoritmos usados.

No.	Time	Source	Destination	Protocol	Length	Info
17	2.252722984	172.17.0.1	172.17.0.2	SSHv2	109	Client: Protocol (SSH-2.0-OpenSSH_7.3p1 Ubuntu-1ubuntu0.1)
21	2.253366065	172.17.0.2	172.17.0.1	SSHv2	109	Server: Protocol (SSH-2.0-OpenSSH_9.0p1 Ubuntu-1ubuntu7.3)
28	2.253847095	172.17.0.1	172.17.0.2	SSHv2	1500	Client: Key Exchange Init
30	2.254503006	172.17.0.2	172.17.0.1	SSHv2	1148	Server: Key Exchange Init
34	2.256561416	172.17.0.1	172.17.0.2	SSHv2	116	Client: Elliptic Curve Diffie-Hellman Key Exchange Init
36	2.261854718	172.17.0.2	172.17.0.1	SSHv2	664	Server: Elliptic Curve Diffie-Hellman Key Exchange Reply, New Keys, Encrypted packet (len=316)
40	2.264302955	172.17.0.1	172.17.0.2	SSHv2	84	Client: New Keys
47	2.305398741	172.17.0.1	172.17.0.2	SSHv2	112	Client: Encrypted packet (len=44)
51	2.305558005	172.17.0.2	172.17.0.1	SSHv2	112	Server: Encrypted packet (len=44)
55	2.305903912	172.17.0.1	172.17.0.2	SSHv2	136	Client: Encrypted packet (len=68)
57	2.313298953	172.17.0.2	172.17.0.1	SSHv2	120	Server: Encrypted packet (len=52)
69	4.798457748	172.17.0.1	172.17.0.2	SSHv2	216	Client: Encrypted packet (len=148)
77	4.856629187	172.17.0.2	172.17.0.1	SSHv2	96	Server: Encrypted packet (len=28)
85	4.8566292160	172.17.0.1	172.17.0.2	SSHv2	180	Client: Encrypted packet (len=112)
89	4.869445655	172.17.0.2	172.17.0.1	SSHv2	696	Server: Encrypted packet (len=628)
95	4.910269539	172.17.0.1	172.17.0.2	SSHv2	112	Server: Encrypted packet (len=44)
102	4.910697257	172.17.0.1	172.17.0.2	SSHv2	444	Client: Encrypted packet (len=376)
104	4.912650032	172.17.0.2	172.17.0.1	SSHv2	176	Server: Encrypted packet (len=108)
107	4.913108082	172.17.0.2	172.17.0.1	SSHv2	568	Server: Encrypted packet (len=500)
113	4.91864988	172.17.0.2	172.17.0.1	SSHv2	104	Server: Encrypted packet (len=36)

Figura 7: Tamaños de paquetes para C1: 1580, 1148, 116 bytes.

```

[hasshAlgorithms [truncated]: curve25519-sha256@libssh.org,ecdh-sha2-nistp256,ecdh-sha2-
[hassh: 0e4584cb9f2dd077dbf8ba0df8112d8e]
Padding String: 000000000000000000000000

```

Figura 8: HASSH para C1

1.4 Tráfico generado por C2, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

1 DESARROLLO (PARTE 1)

1.4. Tráfico generado por C2, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

Aquí se observan tamaños de 1428, 1148 y 116 bytes. Los tipos de algoritmos son más actuales que en C1 y el tráfico revela una negociación de handshake moderna. El HASSH corresponde a las opciones de cifrado detectadas en C2.

No.	Time	Source	Destination	Protocol	Length	Info
140	10.982487069	172.17.0.2	172.17.0.1	SSHv2	109	Server: Protocol (SSH-2.0-OpenSSH_9.0p1 Ubuntu-1ubuntu7.3)
148	10.984365037	172.17.0.1	172.17.0.2	SSHv2	109	Client: Protocol (SSH-2.0-OpenSSH_7.7p1 Ubuntu-4ubuntu0.3)
154	10.985071149	172.17.0.1	172.17.0.2	SSHv2	1428	Client: Key Exchange Init
158	10.986539942	172.17.0.2	172.17.0.1	SSHv2	1148	Server: Key Exchange Init
162	10.990777900	172.17.0.1	172.17.0.2	SSHv2	116	Client: Elliptic Curve Diffie-Hellman Key Exchange Init
164	11.001296755	172.17.0.2	172.17.0.1	SSHv2	664	Server: Elliptic Curve Diffie-Hellman Key Exchange Reply, New Keys, Encrypted packet (len=316)
180	14.177627998	172.17.0.1	172.17.0.2	SSHv2	84	Client: New Keys
187	14.218256976	172.17.0.1	172.17.0.2	SSHv2	112	Client: Encrypted packet (len=44)
191	14.218415683	172.17.0.2	172.17.0.1	SSHv2	112	Server: Encrypted packet (len=44)
198	14.218735551	172.17.0.1	172.17.0.2	SSHv2	136	Client: Encrypted packet (len=68)
200	14.226129052	172.17.0.2	172.17.0.1	SSHv2	120	Server: Encrypted packet (len=52)
219	17.381676242	172.17.0.1	172.17.0.2	SSHv2	216	Client: Encrypted packet (len=148)
224	17.439396136	172.17.0.2	172.17.0.1	SSHv2	96	Server: Encrypted packet (len=28)
232	17.439736149	172.17.0.1	172.17.0.2	SSHv2	180	Client: Encrypted packet (len=112)
236	17.450361385	172.17.0.2	172.17.0.1	SSHv2	696	Server: Encrypted packet (len=628)
242	17.492234153	172.17.0.2	172.17.0.1	SSHv2	112	Server: Encrypted packet (len=44)
249	17.492684444	172.17.0.1	172.17.0.2	SSHv2	444	Client: Encrypted packet (len=376)
251	17.494655875	172.17.0.2	172.17.0.1	SSHv2	176	Server: Encrypted packet (len=108)
254	17.495150172	172.17.0.2	172.17.0.1	SSHv2	568	Server: Encrypted packet (len=500)
260	17.500929894	172.17.0.2	172.17.0.1	SSHv2	104	Server: Encrypted packet (len=36)

Figura 9: Tamaños de paquetes para C2: 1428, 1148, 116 bytes.

```
ncat vcu. vvvvvvvvvv
[hashsh:algorithm] [truncated]: curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2-nistp256,ecdh-
[hashsh: 06046964c022c6407d15a27b12a6a4fb]
Padding String: 0000000000
```

Figura 10: HASSH para C2

1.5. Tráfico generado por C3, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

En C3 destaca el tamaño de paquetes iniciales, como 1500, 1148 y 1276 bytes. Los algoritmos ofrecidos son más robustos y modernos que C1 y C2, reflejando los avances en seguridad. El HASSH lo verifica.

No.	Time	Source	Destination	Protocol	Length	Info
58	10.781326132	172.17.0.2	172.17.0.1	SSHv2	109	Server: Protocol (SSH-2.0-OpenSSH_9.0p1 Ubuntu-1ubuntu7.3)
66	10.784304005	172.17.0.1	172.17.0.2	SSHv2	109	Client: Protocol (SSH-2.0-OpenSSH_8.3p1 Ubuntu-1ubuntu0.1)
72	10.784705971	172.17.0.1	172.17.0.2	SSHv2	1580	Client: Key Exchange Init
76	10.786419523	172.17.0.2	172.17.0.1	SSHv2	1148	Server: Key Exchange Init
80	10.789968877	172.17.0.1	172.17.0.2	SSHv2	116	Client: Elliptic Curve Diffie-Hellman Key Exchange Init
82	10.800164566	172.17.0.2	172.17.0.1	SSHv2	664	Server: Elliptic Curve Diffie-Hellman Key Exchange Reply, New Keys, Encrypted packet (len=316)
99	14.482490899	172.17.0.1	172.17.0.2	SSHv2	84	Client: New Keys
106	14.522962931	172.17.0.1	172.17.0.2	SSHv2	112	Client: Encrypted packet (len=44)
110	14.523140204	172.17.0.2	172.17.0.1	SSHv2	112	Server: Encrypted packet (len=44)
117	14.523441358	172.17.0.1	172.17.0.2	SSHv2	136	Client: Encrypted packet (len=68)
119	14.538795759	172.17.0.2	172.17.0.1	SSHv2	120	Server: Encrypted packet (len=52)
176	17.768711728	172.17.0.1	172.17.0.2	SSHv2	216	Client: Encrypted packet (len=148)
181	17.826449493	172.17.0.2	172.17.0.1	SSHv2	96	Server: Encrypted packet (len=28)
189	17.826788755	172.17.0.1	172.17.0.2	SSHv2	180	Client: Encrypted packet (len=112)
193	17.837580548	172.17.0.2	172.17.0.1	SSHv2	696	Server: Encrypted packet (len=628)
199	17.877747338	172.17.0.2	172.17.0.1	SSHv2	112	Server: Encrypted packet (len=44)
206	17.878931407	172.17.0.1	172.17.0.2	SSHv2	444	Client: Encrypted packet (len=376)
208	17.879916592	172.17.0.2	172.17.0.1	SSHv2	176	Server: Encrypted packet (len=108)
211	17.880349838	172.17.0.2	172.17.0.1	SSHv2	568	Server: Encrypted packet (len=500)
217	17.880951099	172.17.0.2	172.17.0.1	SSHv2	104	Server: Encrypted packet (len=36)

Figura 11: Tamaños de paquetes para C3: 1500, 1148, 1276 bytes.

1.6 Tráfico generado por C4 (iface lo), detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

1 DESARROLLO (PARTE 1)

```
[hasshAlgorithms [truncated]: curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2
[hassh: ae8bd7dd09970555aa4c6ed22adbbf56]
Padding String: 000000000000000000000000
```

Figura 12: HASSH para C3

1.6. Tráfico generado por C4 (iface lo), detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

En el tráfico de C4 se observan tamaños como 1572, 1148, 1276 y 1632 bytes para los paquetes de handshake y clave. Se aprecia el uso de algoritmos modernos y alta variedad, junto con el HASSH correspondiente.

No.	Time	Source	Destination	Protocol	Length	Info
173	14.555661003	172.17.0.1	172.17.0.2	SSHv2	109	Client: Protocol (SSH-2.0-OpenSSH_9.0p1 Ubuntu-1ubuntu7.3)
177	14.556756089	172.17.0.2	172.17.0.1	SSHv2	109	Server: Protocol (SSH-2.0-OpenSSH_9.0p1 Ubuntu-1ubuntu7.3)
184	14.557372085	172.17.0.1	172.17.0.2	SSHv2	1572	Client: Key Exchange Init
188	14.558878951	172.17.0.2	172.17.0.1	SSHv2	1148	Server: Key Exchange Init
195	14.641559019	172.17.0.1	172.17.0.2	SSHv2	1276	Client: Diffie-Hellman Key Exchange Init
197	14.654335782	172.17.0.2	172.17.0.1	SSHv2	1632	Server: Diffie-Hellman Key Exchange Reply, New Keys, Encrypted packet (len=316)
210	19.535628375	172.17.0.1	172.17.0.2	SSHv2	84	Client: New Keys
217	19.575898384	172.17.0.1	172.17.0.2	SSHv2	112	Client: Encrypted packet (len=44)
221	19.576088658	172.17.0.2	172.17.0.1	SSHv2	112	Server: Encrypted packet (len=44)
228	19.576237205	172.17.0.1	172.17.0.2	SSHv2	136	Client: Encrypted packet (len=68)
230	19.583614747	172.17.0.2	172.17.0.1	SSHv2	120	Server: Encrypted packet (len=52)
239	22.224901553	172.17.0.1	172.17.0.2	SSHv2	216	Client: Encrypted packet (len=148)
244	22.282829686	172.17.0.2	172.17.0.1	SSHv2	96	Server: Encrypted packet (len=28)
252	22.283285547	172.17.0.1	172.17.0.2	SSHv2	180	Client: Encrypted packet (len=112)
256	22.305953130	172.17.0.2	172.17.0.1	SSHv2	696	Server: Encrypted packet (len=628)
260	22.306110319	172.17.0.1	172.17.0.2	SSHv2	648	Client: Encrypted packet (len=580)
262	22.306131458	172.17.0.1	172.17.0.2	SSHv2	112	Client: Encrypted packet (len=44)
266	22.306358654	172.17.0.1	172.17.0.2	SSHv2	444	Client: Encrypted packet (len=376)
268	22.313625358	172.17.0.2	172.17.0.1	SSHv2	608	Server: Encrypted packet (len=540)
271	22.315373865	172.17.0.2	172.17.0.1	SSHv2	176	Server: Encrypted packet (len=108)
277	22.315846193	172.17.0.2	172.17.0.1	SSHv2	568	Server: Encrypted packet (len=500)
280	22.321686285	172.17.0.2	172.17.0.1	SSHv2	104	Server: Encrypted packet (len=36)

Figura 13: Tamaños de paquetes para C4: 1572, 1148, 1276, 1632 bytes.

```
[hasshAlgorithms [truncated]: sntrup761x25519-sha512@openssh.com,curve25519-sha256,cur
[hassh: 78c05d999799066a2b4554ce7b1585a6]
Padding String: 00000000
```

Figura 14: HASSH para C4

1.7. Tipo de información contenida en cada uno de los paquetes generados en texto plano

1.7.1. C1

Se visualizan los algoritmos negociados en los paquetes iniciales, antes de que la comunicación cambie a cifrado después del handshake.

1.7.2. C2

Los paquetes iniciales muestran algoritmos de cifrado y autenticación actualizados respecto a C1, aún en texto plano antes del cifrado.

1.7.3. C3

Aparecen nuevos algoritmos y opciones de seguridad adicionales en los primeros paquetes visibles.

1.7.4. C4/S1

La negociación incluye los sets más modernos, mostrando la evolución de protocolos en texto plano antes de aplicar el cifrado.

1.8. Diferencia entre C1 y C2

La diferencia está en los algoritmos soportados y el patrón de handshake, lo que se revela en el tamaño de los paquetes y el fingerprint HASSH.

1.9. Diferencia entre C2 y C3

C3 añade algoritmos más robustos y mejora la seguridad, como se aprecia en el flujo y el HASSH correspondiente.

1.10. Diferencia entre C3 y C4

C4 utiliza las versiones más nuevas de OpenSSH, ofreciendo aún más opciones de cifrado y autenticación, y generando patrones y fingerprints modernos.

2. Desarrollo (Parte 2)

2.1. Identificación del cliente SSH con versión “?”

Al revisar los paquetes, se nota que el intercambio de claves “key exchange init” tiene un tamaño de 1578 bytes, junto con otros valores como 114, 110, 112, 150, entre otros. Esto da pistas sobre qué versión de OpenSSH podría estar usando el informante, ya que cada versión y configuración genera tamaños de paquetes distintos. Para identificar el cliente, se probarían varias versiones de Ubuntu y OpenSSH, aunque en la práctica, el tamaño puede variar según la configuración de algoritmos ofrecidos.

2.2. Replicación de tráfico al servidor (paso por paso)

El objetivo de esta parte es intentar replicar el tráfico visto en la figura anterior. El paso a paso que seguiría es el siguiente:

1. Preparar un contenedor Docker con una versión de Ubuntu que tenga OpenSSH 7.xo cercana debido al cercanismo que tiene con el tamaño del “key exchange init”
2. Realizar una conexión SSH hacia el servidor y capturar los paquetes usando Wireshark.
3. Revisar los tamaños de los paquetes clave, especialmente el “key exchange init”, y compararlos con los valores de la imagen del informante (buscando igualar los 1578 bytes y el resto de la estructura de paquetes).
4. En caso de que no coincida el tamaño, probar con otras versiones.

Por razones de tiempo y complejidad, no me fue posible probar todas las combinaciones posibles.

3. Desarrollo (Parte 3)

3.1. Replicación del KEI con tamaño menor a 300 bytes (paso por paso)

Para reducir el tamaño del Key Exchange Init (KEI) en el servidor y dejarlo bajo los 300 bytes, se deben limitar los algoritmos que el servidor oferta en el handshake SSH. Los pasos son los siguientes:

1. Accedí al archivo de configuración del servidor SSH `/etc/ssh/sshd_config` y edité las líneas relacionadas con los algoritmos, manteniendo solo una opción por categoría:

```
KexAlgorithms curve25519-sha256  
Ciphers aes128-ctr
```

```
MACs hmac-sha1
HostKeyAlgorithms ssh-ed25519
```

Esto reduce la cantidad de datos que el servidor envía en el KEI al ofertar menos algoritmos.

2. Guardé el archivo y reinicié el servicio SSH para aplicar los cambios.

```
service ssh restart
```

3. Realicé nuevamente una conexión SSH hacia el servidor y capturé el tráfico con Wireshark.
4. Verifiqué en la captura que el paquete Key Exchange Init enviado por el servidor tuviera un tamaño menor a 300 bytes.

Time	Source	Destination	Protocol	Length	Info
87.9.323710272	172.17.0.1	172.17.0.2	SSHv2	189	Client: Protocol (SSH-2.0-OpenSSH_8.3p1 Ubuntu-1ubuntu0.1)
91.9.324269443	172.17.0.2	172.17.0.1	SSHv2	189	Server: Protocol (SSH-2.0-OpenSSH_9.0p1 Ubuntu-1ubuntu7.3)
98.9.324488980	172.17.0.1	172.17.0.2	SSHv2	1580	Client: Key Exchange Init
102.9.325376353	172.17.0.2	172.17.0.1	SSHv2	252	Server: Key Exchange Init
106.9.327025926	172.17.0.1	172.17.0.2	SSHv2	416	Client: Elliptic Curve Diffie-Hellman Key Exchange Init

Figura 15: Key Exchange Init <300 de tamaño

Con este procedimiento, se logra que el KEI tenga un tamaño reducido a menos de 300.

4. Desarrollo (Parte 4)

4.1. Explicación OpenSSH en general

OpenSSH es una versión libre y segura del protocolo SSH, usada para controlar y administrar servidores o dispositivos de manera remota por internet[web:40][web:41]. SSH funciona usando un modelo cliente-servidor, donde el cliente inicia la conexión y el servidor la recibe. El objetivo principal de OpenSSH es asegurar que los datos viajen cifrados entre los dos puntos, evitando que terceros puedan leer la información. Para conectarse, el cliente y el servidor acuerdan los algoritmos de cifrado y autenticación, y luego el usuario se autentica usando una clave o contraseña[web:40][web:44][web:41].

4.2. Capas de Seguridad en OpenSSH

En OpenSSH existen varias capas de seguridad que protegen los datos y la identidad del usuario:

- **Confidencialidad:** Todos los datos enviados entre el cliente y el servidor son cifrados, usando algoritmos como AES o `aes128-ctr`, para que nadie más pueda leer el contenido entre ambos equipos.
- **Integridad:** Utiliza mecanismos de comprobación como MACs, por ejemplo `hmac-sha1`, para asegurarse de que los datos no hayan sido modificados mientras viajan por la red.
- **Autenticidad:** El cliente verifica la identidad del servidor (por ejemplo, aceptando la huella digital) y a su vez el usuario puede autenticarse con clave o contraseña; esto asegura que ambas partes son quienes dicen ser.
- **Disponibilidad:** Aunque OpenSSH permite configurar reglas de acceso y restringir algoritmos débiles, la disponibilidad depende más del sistema donde está instalado que del protocolo mismo. SSH no protege directamente contra denegaciones de servicio, pero ayuda a controlar accesos y evitar intrusos.
- **No repudio:** OpenSSH registra los intentos de conexión y autenticidad, sin embargo, si las claves o contraseñas son compartidas no se puede garantizar completamente el no repudio, es decir, demostrar quién realmente realizó una acción; los registros y las claves ayudan, pero no hacen que el principio se cumpla de forma absoluta.

4.3. Identificación de que protocolos no se cumplen

Basándonos en el análisis del tráfico capturado y la configuración usada en el laboratorio, identificamos que:

- El principio de no repudio no se cumple completamente, debido a que OpenSSH no implementa mecanismos que aseguren la trazabilidad absoluta de las acciones y su vinculación inequívoca a un usuario. La compartición de claves puede permitir que un usuario niegue responsabilidades.
- La confidencialidad es parcial durante la fase inicial de negociación, en donde se transmiten en texto plano ciertos mensajes (como la versión del cliente y la lista de algoritmos soportados), exponiendo metadatos que pueden ser explotados para ataques de fingerprinting o vulnerabilidades específicas.

En resumen, OpenSSH ofrece un protocolo seguro con múltiples capas que protegen la mayoría de los principios esenciales, aunque ciertos aspectos como el no repudio y la exposición inicial de metadatos merecen atención y consideración adicionales.

Conclusiones y comentarios

Este laboratorio destacó la importancia de la configuración práctica y controlada del protocolo OpenSSH para asegurar el tráfico SSH. El uso de contenedores Docker con versiones

específicas permitió simular y replicar condiciones reales del protocolo, facilitando la experimentación con diferentes configuraciones de algoritmos y seguridad. La posibilidad de ajustar el servidor SSH mediante la modificación directa del archivo `sshd_config` y revisar el impacto en el tráfico fue clave para comprender cómo cada parámetro influye en la seguridad y el comportamiento del protocolo.

Repositorio github:<https://github.com/Xxfelipe89xX/Lab05-Cripto>